UNIT – 3 INTRODUCTIONS TO R AND WORKING WITH DATA

1. Overview of R and its Applications in Data Analysis and Statistics

Overview of R

R is a programming language and software environment designed primarily for statistical computing and graphics. It was created by **statisticians Ross Ihaka and Robert Gentleman in 1993** and has since become one of the most popular tools for **data analysis, statistics, and data visualization. R is open-source** and maintained by the R Development Core Team and the R Foundation for Statistical Computing.

Applications of R

1. **Data Analysis:** R is widely used for data cleaning, transformation, and analysis. It provides a rich set of tools and packages for handling and manipulating data.
2. **Statistical Analysis:** R supports a broad range of statistical techniques, including linear and nonlinear modeling, classical statistical tests, time-series analysis, classification, and clustering.
3. **Data Visualization:** R has strong graphical capabilities, allowing users to create static and interactive graphics, including plots, charts, and graphs.
4. **Machine Learning:** R includes numerous packages for machine learning, enabling users to build and evaluate predictive models.
5. **Bioinformatics:** R is extensively used in bioinformatics for analyzing biological data.
6. **Econometrics:** Economists use R for economic modeling and analysis.
7. **Finance:** R is applied in quantitative finance for risk analysis, portfolio optimization, and time-series analysis.

2. Installing R and RStudio

Installing R

1. Download R: Visit the Comprehensive R Archive Network (CRAN) website at https://cran.r-project.org/.

2. Choose Your OS: Select your operating system (Windows, macOS, or Linux).

3. Download Installer: Download the appropriate installer file for your operating system.

4. Run Installer: Run the installer and follow the on-screen instructions to complete the installation.

Installing RStudio

1. **Download RStudio:** Visit the RStudio website at https://www.rstudio.com/.
2. **Choose RStudio Desktop:** Select the free RStudio Desktop version.
3. **Download Installer:** Download the installer file for your operating system.
4. **Run Installer:** Run the installer and follow the on-screen instructions to complete the installation.

3. Basic R Syntax, Variables, and Data Types

Basic R Syntax

Comments: Use # to add comments. For example: # This is a comment.

Assignment: Use `<-` or `=` to assign values to variables. For example: `x <- 10` or `x = 10`.

Print: Use `print()` to print values. For example: `print(x)`.

Variables

Variables in R can store different types of data and are case-sensitive. Variable names can include letters, numbers, dots, and underscores but must start with a letter or a dot.

Data Types

Numeric: Represents numbers. For example: `x <- 10.5`.

Integer: Represents integer values. Use `L` to specify an integer. For example: `y <- 10L`.

Character: Represents text strings. For example: `z <- "Hello, World!"`.

Logical: Represents Boolean values `TRUE` or `FALSE`. For example: `flag <- TRUE`.

Factor: Categorical data. For example: `factor_var <- factor(c("Male", "Female"))`.

Arithmetic Operators

Operator	Description
+	addition
-	subtraction
*	multiplication
/	division
^ or **	exponentiation
x %% y	modulus (x mod y) 5%%2 is 1
x %/% y	integer division 5%/%2 is 2

Logical Operators

Operator	Description
<	less than
<=	less than or equal to
>	greater than
>=	greater than or equal to
==	exactly equal to
!=	not equal to
!x	Not x
x | y	x OR y
x & y	x AND y
isTRUE(x)	test if x is TRUE

Numeric Functions

Function	Description
abs(*x*)	absolute value
sqrt(*x*)	square root
ceiling(*x*)	ceiling(3.475) is 4
floor(*x*)	floor(3.475) is 3
trunc(*x*)	trunc(5.99) is 5
round(*x*, **digits**=*n*)	round(3.475, digits=2) is 3.48
signif(*x*, **digits**=*n*)	signif(3.475, digits=2) is 3.5
cos(*x*), **sin**(*x*), **tan**(*x*)	also acos(*x*), cosh(*x*), acosh(*x*), etc.
log(*x*)	natural logarithm
log10(*x*)	common logarithm
exp(*x*)	e^*x*

Character Functions

Function	Description
substr(*x***, start=***n1***, stop=***n2***)**	Extract or replace substrings in a character vector. x <- "abcdef" substr(x, 2, 4) is "bcd" substr(x, 2, 4) <- "22222" is "a222ef"
grep(*pattern***,***x* **, ignore.case**=FALSE, **fixed**=FALSE)	Search for *pattern* in *x*. If fixed =FALSE then *pattern* is a regular expression. If fixed=TRUE then *pattern* is a text string. Returns matching indices. grep("A", c("b","A","c"), fixed=TRUE) returns 2
sub(*pattern***,***replacement***,***x***, ignore.case** =FALSE, **fixed**=FALSE)	Find *pattern* in *x* and replace with *replacement* text. If fixed=FALSE then *pattern* is a regular expression. If fixed = T then *pattern* is a text string. sub("\\s",".","Hello There") returns "Hello.There"
strsplit(*x***,** *split***)**	Split the elements of character vector *x* at *split*. strsplit("abc", "") returns 3 element vector "a","b","c"
paste(..., sep="")	Concatenate strings after using *sep* string to seperate them. paste("x",1:3,sep="") returns c("x1","x2" "x3") paste("x",1:3,sep="M") returns c("xM1","xM2" "xM3") paste("Today is", date())
toupper(*x***)**	Uppercase
tolower(*x***)**	Lowercase

Statistics Functions

Function	Description
mean(*x***, trim**=0, **na.rm**=FALSE)	mean of object x # trimmed mean, removing any missing values and # 5 percent of highest and lowest scores mx <- mean(x,trim=.05,na.rm=TRUE)
sd(*x***)**	standard deviation of object(x). also look at var(x) for variance and mad(x) for median absolute deviation.
median(*x***)**	median
quantile(*x***,** *probs***)**	quantiles where x is the numeric vector whose quantiles are desired and probs is a numeric vector with probabilities in [0,1]. # 30th and 84th percentiles of x y <- quantile(x, c(.3,.84))
range(*x***)**	range
sum(*x***)**	sum
diff(*x***, lag=***1***)**	lagged differences, with lag indicating which lag to use
min(*x***)**	minimum
max(*x***)**	maximum
scale(*x***, center**=TRUE, **scale**=TRUE)	column center or standardize a matrix.

4. Importing Data into R from Different File Formats (CSV, Excel, etc.)

Importing CSV Files

Use the `read.csv()` function to import CSV files.

Import CSV file

```
data <- read.csv("path/to/your/file.csv")
```

Importing Excel Files

Use the `readxl` package to import Excel files. Install the package using `install.packages("readxl")` and then use the `read_excel()` function.

Install and load readxl package

```
install.packages("readxl")
library(readxl)
```

Import Excel file

```
data <- read_excel("path/to/your/file.xlsx")
```

Importing Other Formats

R supports importing data from various formats, including text files, JSON, and SQL databases. For example, to import data from a text file, you can use the `read.table()` function.

Import text file

```
data <- read.table("path/to/your/file.txt", header = TRUE, sep = "\t")
```

5. Viewing and Inspecting Data Frames

Viewing Data Frames

Head and Tail: Use `head()` and `tail()` to view the first and last few rows of the data frame.

View the first 6 rows

head(data)

View the last 6 rows

tail(data)

Summary: Use `summary()` to get a summary of the data frame.

Get summary statistics

summary(data)

Structure: Use `str()` to view the structure of the data frame.

View structure of the data frame

str(data)

Inspecting Data Frames

Column Names: Use `colnames()` to get or set the column names.

Get column names

colnames(data)

Dimensions: Use `dim()` to get the dimensions (number of rows and columns) of the data frame.

Get dimensions

dim(data)

Data Types: Use `sapply()` to get the data types of each column.

Get data types of each column

sapply(data, class)

CHAPTER SUMMARY

Basics of R Programming

- **What is R?**
- R Programming is used as a leading tool for machine learning, statistics, and data analysis. Objects, functions, and packages can easily be created by R.
- It's a platform-independent language. That means it can be applied to all operating systems.
- It's an open-source free language. That means anyone can install it in any organization without purchasing a license.
- R programming language is not only a statistics package but also allows us to integrate with other languages (C, C++). Thus, you can easily interact with many data sources and statistical packages.
- The R programming language has a vast community of users and it's growing day by day.
- R is currently one of the most requested programming languages in the Data Science job market which makes it the hottest trend nowadays.
- It was designed by Ross Ihaka and Robert Gentleman at the University of Auckland, New Zealand and is currently being developed by the R Development Core Team.
- R programming language is an implementation of the S programming language. It also combines with lexical scoping semantics inspired by Scheme. Moreover, the project was conceived in 1992, with an initial version released in 1995 and a stable beta version in 2000.

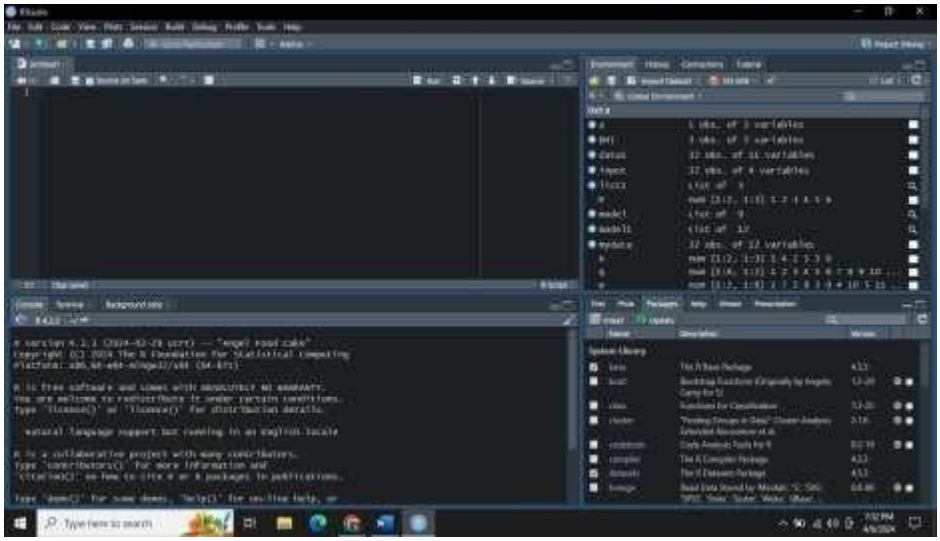
- **Why Use R?**
- **Statistical Analysis:** R is designed for analysis and it provides an extensive collection of graphical and statistical techniques, by making a preferred choice for statisticians and data analysts.
- **Open Source:** R is an open – source software, which means it is freely available to anyone. It can be accessible by a vibrant community of users and developers.
- **Data Visualization:** R boasts an array of libraries like ggplot2 that enable the creation of high-quality, customizable data visualizations.
- **Data Manipulation:** R offers tools that are for data manipulation and transformation. For example: IT simplifies the process of filtering, summarizing and transforming data.
- **Integration:** R can be easily integrated with other programming languages and data sources. IT has connectors to various databases and can be used in conjunction with python, SQL and other tools.
- **Community and Packages:** R has vast ecosystem of packages that extend its functionality. There are packages that can help you accomplish needs of analytics.

- **Advantages of R**
- R is the most comprehensive statistical analysis package. As new technology and concepts often appear first in R.
- As R programming language is an open source. Thus, you can run R anywhere and at any time.
- R programming language is suitable for GNU/Linux and Windows operating systems.

- R programming is cross-platform and runs on any operating system.
- In R, everyone is welcome to provide new packages, bug fixes, and code enhancements.

- **Disadvantages of R**
- In the R programming language, the standard of some packages is less than perfect.
- Although, R commands give little pressure on memory management. So R programming language may consume all available memory.
- In R basically, nobody to complain if something doesn't work.
- R programming language is much slower than other programming languages such as Python and MATLAB.

- **What is R Studio**
- R Studio is an integrated development environment (IDE) for R. IDE is a GUI, where you can write your quotes, see the results and also see the variables that are generated during the course of programming.
- R Studio can be downloaded from its official website (https://rstudio.com/).
- After the installation process is over, the R Studio interface looks like:



- The console panel (bottom left panel) is the place where R is waiting for you to tell it what to do, and see the results that are generated when you type in the commands.
- The top left pane is where you write the code and queries in order to get the results.
- To the top right, you have the Environmental/History panel. It contains 2 tabs:
- **Environment tab:** It shows the variables that are generated during the course of programming in a workspace that is temporary.
- **History tab:** In this tab, you'll see all the commands that are used till now from the start of usage of R Studio.
- To the right bottom, you have another panel, which contains multiple tabs, such

as files, plots, packages, help, and viewer.

- The **Files tab** shows the files and directories that are available within the default workspace of R.
- The **Plots tab** shows the plots that are generated during the course of programming.
- The **Packages** tab helps you to look at what are the packages that are already installed in the R Studio and it also gives a user interface to install new packages.

- The Help **tab** is the most important one where you can get help from the R Documentation on the functions that are in built-in R.
- The final and last tab is that the **Viewer** tab which can be used to see the local web content that's generated using R.

- Syntax in R.
- A program in R is made up of three things: **Variables, Comments, and Keywords**. **Variables** are used to store the data, **Comments** are used to improve code readability, and **Keywords** are reserved words that hold a specific meaning to the compiler.

  - **Variables in R**
  - Previously, we wrote all our code in a print() but we don't have a way to address them as to perform further operations. This problem can be solved by using variables which like any other programming language are the name given to reserved memory locations that can store any type of data. In R, the assignment can be denoted in three ways:
  - = (Simple Assignment)
  - <- (Leftward Assignment)

  - **Comments in R**
  - Comments are a way to improve your code's readability and are only meant for the user so the interpreter ignores it. Only single-line comments are available in R but we can also use multiline comments by using a simple trick which is shown below. Single line comments can be written by using # at the beginning of the statement.

  - **Keywords in R**
  - Keywords are the words reserved by a program because they have a special meaning thus a **keyword can't be used as a variable name, function name, etc**. We can view these keywords by using either help(reserved) or ?reserved.
  - if, else, repeat, while, function, for, in, next and break are used for control-flow statements and declaring user-defined functions.
  - The ones left are used as constants like TRUE/FALSE are used as Boolean constants.
  - NaN defines Not a Number value and NULL are used to define an Undefined value.
  - Inf is used for Infinity values.

- R Operators
- Operators are the symbols directing the compiler to perform various kinds of operations between the operands. Operators simulate the various mathematical, logical, and decision operations performed on a

set of Complex Numbers, Integers, and Numerical as input operands.

- R supports majorly four kinds of binary operators between a set of operands. In this article, we will see various types of operators in R Programming language and their usage.
- Types of the operator in R language
    - Arithmetic Operators

        - **Addition Operator (+)**
        - Example: -

```
a <- c (1, 0.1)
b <- c (2.33, 4)
print (a+b)
```

        - **Subtraction Operator (-)**

- Example: -

```
a <- c (1, 0.1)

b <- c (2.33, 4)

print (a-b)
```

- **Multiplication Operator (*)**
- Example: -

```
a <- c (1, 0.1)

b <- c (2.33, 4)

print (a*b)
```

- **Division Operator (/)**
- Example: -

```
a <- c (20, 100)

b <- c (2, 4) print (a/b)
```

- **Power Operator (^)**
- Example: -

```
a <- 4

b <-3

print (a^b)
```

- **Modulo Operator (%%)**
- Example: -

```
a <- c (2, 10)

b <- c (2, 4) print (a%%b)
```

 # R program to illustrate

```
# the use of Logical operators
vec1 <- c(0,2)

vec2 <- c(TRUE,FALSE)


# Performing operations on Operands

cat ("Element wise AND :", vec1 & vec2,
"\n") cat ("Element wise OR :", vec1 |
vec2, "\n") cat ("Logical AND :", vec1[1]
&& vec2[1], "\n")

cat ("Logical OR :", vec1[1] || vec2[1],
"\n") cat ("Negation:", !vec1)
```

- Relational Operators
  - Less than (<)
  - Greater than (>)
  - Less than equals to (<=)
  - Greater than equals to (>=)
  - Not equal to (!=)
  - **Examples: -**

```
# R program to illustrate

# the use of Relational
operators vec1 <- c(0, 2)

vec2 <- c(2, 3)


# Performing operations on Operands

cat ("Vector1 less than Vector2 :", vec1 < vec2, "\n")

cat ("Vector1 less than equal to Vector2 :", vec1 <= vec2,
"\n") cat ("Vector1 greater than Vector2 :", vec1 >
vec2, "\n")

cat ("Vector1 greater than equal to Vector2 :", vec1 >= vec2,
"\n") cat ("Vector1 not equal to Vector2 :", vec1 != vec2,
"\n")
```

- Assignment Operators
  - Left Assignment (<- or <<- or =)
  - Right Assignment (-> or ->>)
  - **Examples: -**

```
# R program to illustrate
```

```
# the use of Assignment operators
vec1 <- c(2:5)

c(2:5) ->>
vec2 vec3 <<-
c(2:5) vec4 =
c(2:5) c(2:5) -
> vec5


# Performing operations on
Operands cat ("vector 1 :", vec1,
"\n")

cat("vector 2 :", vec2, "\n")

cat ("vector 3 :", vec3, "\n")

cat("vector 4 :", vec4, "\n")
cat("vector 5 :", vec5)
```

- Miscellaneous Operators
  - In Operator (%in%)
    Checks if an element belongs to a list and returns a Boolean value TRUE if the value is present else FALSE.
  - **Example: -**

```
val <- 0.1

list1 <- c(TRUE,
0.1,"apple") print (val
%in% list1)
```

- **Data Types in R.**
- Each variable in R has an associated data type. Each R-Data Type requires different amounts of memory and has some specific operations which can be performed over it.
- Data Types in R are:
  - **Numeric – (3,6.7,121)**
    - Examples: -

```
1. # A simple R program
   # to illustrate Numeric data
   type # Assign a decimal value
   to x
   x = 5.6
   # print the class name of
   variable print(class(x))
```

```
# print the type of variable
print(typeof(x))
```

2.
```
# A simple R program
# to illustrate Numeric data
type # Assign an integer value
to y
y = 5
# print the class name of
variable print(class(y))
# print the type of variable
print(typeof(y))
```

- **Integer – (2L, 42L; where 'L' declares this as an integer**)
  - Examples: -
    1.
```
# A simple R program
# to illustrate integer data
type # Create an integer
value
x = as.integer(5)
# print the class name of x
print(class(x))
# print the type of
x print(typeof(x))
# Declare an integer by appending an L suffix.


y = 5L
# print the class name of y
print(class(y))
# print the type of
y print(typeof(y))
```

- **logical – ('True')**
  - Examples: -
    1.
```
# A simple R program
# to illustrate logical data
type # Sample values
x = 4
y = 3
# Comparing two
values z = x > y
# print the logical
value print(z)
# print the class name of z
print(class(z))
# print the type of
```

```
z print(typeof(z))
```

- **complex – (7 + 5i; where 'i' is imaginary number)**
  - Examples: -

```
1. # A simple R program
# to illustrate complex data
type # Assign a complex
value to x
x = 4 + 3i
# print the class name of x
print(class(x))
# print the type of
x print(typeof(x))
```

- **character – ("a", "B", "c is third", "69")**
  - Example: -

```
1. # A simple R program
# to illustrate character data
type # Assign a character value
to char char = "Geeksforgeeks"
# print the class name of char
print(class(char))
# print the type of char
print(typeof(char))
```

- **Find datatype of your object in R**
  - Example: -
    - # A simple R program

```
# to find data type of an
object # Logical
print(class(TRU
E)) # Integer
print(class(3L))
# Numeric
print(class(10.5)
) # Complex
print(class(1+2i
)) # Character print(class("12-04-
2020"))
```

- **Convert the Data Type of an Object to Another**
  - Example: -
    - # A simple R program

```
# convert data type of an object to
another # Logical
print(as.numeric(TRU
```

```
E)) # Integer
print(as.complex(3L))
# Numeric
print(as.logical(10.5
)) # Complex
print(as.character(1+2i
)) # Can't possible
print(as.numeric("12-04-2020"))
```

- **Data Structures in R.**
- A data structure is a particular way of organizing data in a computer so that it can be used effectively.
- The idea is to reduce the space and time complexities of different tasks.
- The most essential data structures used in R include:
  - **Vectors**: - A vector is an ordered collection of basic data types of a given length. The only key thing here is all the elements of a vector must be of the identical data type e.g homogeneous data structures. Vectors are one-dimensional data structures.
    - **Example: -**

```
# R program to illustrate Vector

# Vectors(ordered collection of same data
type) X = c(1, 3, 5, 7, 8)

# Printing those elements in
console print(X)
```

  - **Lists: -**
    - A list is a generic object consisting of an ordered collection of objects.
    - Lists are heterogeneous data structures.
    - These are also one-dimensional data structures.
    - A list can be a list of vectors, list of matrices, a list of characters and a list of functions and so on.
    - **Examples: -**

```
# R program to illustrate a List

# The first attributes is a numeric
vector # containing the employee
IDs which is # created using the 'c'
command here empId = c(1, 2, 3, 4)

# The second attribute is the employee
name # which is created using this line of
code here # which is the character vector
```

```
empName = c("Debi", "Sandeep", "Subham",
"Shiba") # The third attribute is the number of
employees

# which is a single numeric variable.
numberOfEmp = 4

# We can combine all these three
different # data types into a list

# containing the details of employees

# which can be done using a list command
empList = list(empId, empName,
numberOfEmp) print(empList)
```

- **Dataframes: -**
  - Dataframes are generic data objects of R which are used to store the tabular data.
  - Dataframes are the foremost popular data objects in R programming because we are comfortable in seeing the data within the tabular form.
  - They are two-dimensional, heterogeneous data structures.
  - **Example: -**

```
# R program to illustrate dataframe

# A vector which is a character
vector Name = c("Amiya",
"Raj", "Asish")

# A vector which is a character
vector Language = c("R",
"Python", "Java")  # A vector
which is a numeric vector Age =
c(22, 25, 45)

# To create dataframe use data.frame
command # and then pass each of the
vectors

# we have created as
arguments # to the function
data.frame()

df = data.frame(Name, Language, Age)
print(df)
```

- **Matrices: -**
  - A matrix is a rectangular arrangement of numbers in rows and columns.
  - Matrices are two-dimensional, homogeneous data structures.

- **Example: -**

  Mat1 = matrix(c(1:10), nrow=2, ncol = 5,byrow=true)
  Print(Mat1)

- **Array: -**
  - Arrays are the R data objects which store the data in more than two dimensions.
  - Arrays are n-dimensional data structures.
  - For example, if we create an array of dimensions (2, 3, 3) then it creates 3 rectangular matrices each with 2 rows and 3 columns.
  - They are homogeneous data structures**.**
  - **Example: -**

    Arr= array(c(a:b), dim=c(rows, columns, no. of matrices)) Arr= array(c(1:20), dim=c(2,5,2))

- **Factors: -**
  - **Factors are the data objects which are used to categorize the data and store it as levels.**
  - **They can store both strings and integers.**
  - **They are useful to categorize unique values in columns like "TRUE" or "FALSE", or "MALE" or "FEMALE", etc.**
  - **They are useful in data analysis for statistical modeling.**
  - **Example: -**

    # R program to illustrate factors # Creating factor using factor()

    fac = factor(c("Male", "Female", "Male","Male", "Female", "Male", "Female")) print(fac)

- **Importing different from various sources**
- In R, you can import data from various file formats including CSV, Excel, SAS, and DAT files. Here's how you can do it for each format:
  - **<u>Importing CSV Files</u>**
  - Example: -
    # Load the data from a CSV file data <- read.csv("your_file.csv")

  - **<u>Importing Excel Files</u>**
  - You can use the `readxl` package to import Excel files.
  - Examples: -

```
# Install and load the readxl
package
install.packages("readxl")
library(readxl)

# Load the data from an Excel
file data <-
read_excel("your_file.xlsx")
```

- **Importing SAS Files**
- You can use the `haven` package to import SAS files.
- Example: -

```
# Install and load the haven
package install.packages("haven")
library(haven)

# Load the data from a SAS file

data <- read_sas("your_file.sas7bdat")
```

- **Importing DAT Files**
- The method for importing DAT files might depend on the specific structure of the data within the file. You might need to adjust the parameters of the importing function accordingly.
- Example: -

```
# Load the data from a DAT file using read.table (assuming it's a tab-
delimited file) data <- read.table("your_file.dat", header = TRUE, sep =
"\t")
```

Note: -Adjust the parameters of the functions (`read.csv`, `read_excel`, `read_sas`, `read.table`) according to your specific data file's structure and requirements.

UNIT 3 (SHORT QUESTIONS AND ANSWERS)

1. Introduction to R and working with Data

- **QUESTION: What is R?**
  - ANSWER: R is a free, open-source programming language and environment for statistical computing and graphics. It's widely used for data analysis, scientific computing, and machine learning. For example, you can use R to analyze survey data, build predictive models, or visualize complex datasets.

2. Overview of R and its applications in data analysis and statistics

- **QUESTION: What are some key applications of R in data analysis?**
  - ANSWER: R is used for a vast array of data analysis tasks. It excels in statistical modeling (linear regression, ANOVA, time series), data visualization (histograms, scatter plots, box plots), and machine learning (classification, clustering, prediction). For instance, you can use R to analyze customer behavior data, predict stock prices, or identify patterns in biological data.

3. Installing R and RStudio

- **QUESTION: Why is RStudio recommended for R programming?**
  - ANSWER: RStudio provides a user-friendly interface for R, making it easier to write, test, and debug code. It offers features like syntax highlighting, code completion, and integrated plotting. For example, if you're new to R, RStudio's interactive environment can help you learn and explore the language more efficiently.

4. Basic R syntax, variables, and data types

- **QUESTION: Explain the concept of variables in R with an example.**
  - ANSWER: Variables in R are used to store data values. They are created using the assignment operator <-. For example, age <- 30 assigns the value 30 to the variable age. R supports various

data types like numeric (e.g., 3.14), character (e.g., "Hello"), logical (e.g., TRUE), and integer (e.g., 5L).

5. Importing data into R from different file formats (CSV, Excel, etc.)

- **QUESTION: How to import a CSV file into R with an example?**
  - ANSWER: The read.csv() function is used to import CSV files into R. For example, data <- read.csv("mydata.csv") imports a CSV file named "mydata.csv" into a data frame called "data". You can customize the import process using arguments like header, sep, and na.strings to handle specific file formats.

6. Viewing and inspecting data frames

- **QUESTION: How to get a summary of a data frame with an example?**
  - ANSWER: The summary() function provides a statistical summary of a data frame. For example, summary(mydata) will display summary statistics for each column in the data frame "mydata", including count, mean, median, quartiles, minimum, and maximum values.

MCQ'S of Unit – 3

Introduction to R and Working with Data

1. **What is R primarily used for?**
   - a) Word processing
   - b) Statistical computing and graphics
   - c) Image editing
   - d) Web development
   - **Answer: b**
2. **Which of the following is NOT a key feature of R?**
   - a) Open-source
   - b) Large community
   - c) Proprietary software
   - d) Extensive package ecosystem

- **Answer: c**

Installing R and RStudio

3. **RStudio is primarily a(n):**
- a) Programming language
- b) Statistical software
- c) Integrated Development Environment (IDE) for R
- d) Data visualization tool
- **Answer: c**
4. **Which of the following is NOT a component of the RStudio interface?**
- a) Console
- b) Script editor
- c) Environment pane
- d) Word processor
- **Answer: d**

Basic R syntax, variables, and data types

5. **Which data type is used for logical values (TRUE or FALSE) in R?**
- a) Numeric
- b) Character
- c) Logical
- d) Integer
- **Answer: c**
6. **What is the correct way to assign the value 10 to a variable named x in R?**
- a) x = 10
- b) 10 -> x
- c) x <- 10
- d) x == 10
- **Answer: c**
7. **Which of the following is NOT a basic data type in R?**
- a) Numeric

- b) Character
- c) Logical
- d) Array
- **Answer: d**

Importing data into R from different file formats (CSV, Excel, etc.)

8. **Which function is commonly used to read CSV files into R?**
- a) read_excel()
- b) read.csv()
- c) import.data()
- d) load.data()
- **Answer: b**
9. **Which R package is typically used for reading Excel files?**
- a) readr
- b) dplyr
- c) readxl
- d) tidyverse
- **Answer: c**

Viewing and inspecting data frames

10. **How can you view the first few rows of a data frame in R?**
- a) head()
- b) tail()
- c) summary()
- d) describe()
- **Answer: a**
11. **Which function provides a summary of a data frame, including statistics for each column?**
- a) head()
- b) tail()
- c) summary()

- d) str()
- **Answer: c**